Темы занятия

• Виды модификаторов доступа
• Модификаторы доступа в рамках проекта
• Модификаторы доступа в рамках сборки
• Файл как область видимости
• Целостность данных и инкапсуляция
• Принципы SOLID: SRP, OCP, LSP, ISP, DIP

Домашние задания

Постановка

1. Выполните **индивидуально Задание** 1.
2. Выполните **индивидуально Задание** 2.
3. Выполните **совместно** в рабочей группе **Задание** 3.
4. Сформулируйте **список вопросов**, возникших в ходе выполнения **заданий** 1, 2 и 3.
5. **Передайте** список вопросов **Лидеру** группы.
6. (**Выполняет Лидер группы**) **Оформите Отчёт** о проделанной работе. Прикрепите список вопросов от каждого участника рабочей группы к Отчёту. (Требования к отчёту см. ниже.)
7. (**Выполняет Лидер группы**) **Отправьте Отчёт** о проделанной работе на указанную на занятии электронную почту.

Задание 1

Задание для разминки. Приведите в порядок описание класса `Student` : закройте поля и оставьте снаружи только проверяющие методы (см. `//TODO: ⚠️`).

```
public class Student
{
    //TODO: ⚠️ Сделайте поля закрытыми и добавьте проверку в конструктор ⚠️
    public string firstName;   // имя
    public string lastName;    // фамилия
    public int age;            // возраст (14..100)
    public double averageMark; // средний балл (0..5)

    public Student(string firstName, string lastName, int age, double
```

```
averageMark)
    {
        //TODO: ⚠️ Проверить все данные и присвоить поля ⚠️
    }

    //TODO: ⚠️ public string GetFullName() ⚠️
    //TODO: ⚠️ public int GetAge() ⚠️
    //TODO: ⚠️ public double GetAverageMark() ⚠️
    //TODO: ⚠️ public void SetAge(int newAge) с проверкой диапазона ⚠️
    //TODO: ⚠️ public string GetInfo() — одна строка со всеми данными ⚠️
}
```

Проверьте:

1. Попытка создать студента с возрастом `-3` должна приводить к исключению.
2. Попытка обратиться к `student.age` из другого класса не должна компилироваться.
3. `SetAge(200)` не должен менять возраст.
4. Попытка создать студента с пустой фамилией должна приводить к исключению.

> ℹ️ Свойства (короткая запись `{ get; set; }`) мы подробно разберём **на следующем занятии**. В этой работе достаточно закрытых полей и открытых методов — так виднее, где именно находится проверка.

Альтернативный вариант (по желанию): тот же класс на автосвойствах. Если хочется заранее попробовать свойства — сделайте так: проверки остаются внутри класса, но доступ теперь организуют **модификаторы доступа у аксессоров**, а это уже тема сегодняшнего занятия.

```
public class Student
{
    // Автосвойства: короткая запись, значение задаётся только в
конструкторе
    public string FirstName { get; }
    public string LastName { get; }

    // Свойство для чтения и записи: сеттер закрыт, снаружи доступно только
чтение
    public int Age { get; private set; }

    public double AverageMark { get; }

    public Student(string firstName, string lastName, int age, double
averageMark)
    {
```

```
        EnsureNotEmpty(firstName, nameof(firstName));
        EnsureNotEmpty(lastName, nameof(lastName));
        EnsureValidAge(age);

        FirstName = firstName;
        LastName = lastName;
        Age = age;
        AverageMark = averageMark;
    }

    // Вычисляемое свойство (подробно — на следующем занятии)
    public string FullName => $"{FirstName} {LastName}";

    // Единственный способ изменить возраст — через проверку
    public void SetAge(int newAge)
    {
        EnsureValidAge(newAge);
        Age = newAge;
    }

    public string GetInfo() => $"{FullName}, {Age} лет, средний балл:
{AverageMark:0.0}";

    // 1-2 приватных метода проверки — они и есть деталь реализации
    private static void EnsureValidAge(int age)
    {
        if (age < 14 || age > 100)
            throw new ArgumentOutOfRangeException(nameof(age), "Возраст
должен быть от 14 до 100");
    }

    private static void EnsureNotEmpty(string value, string fieldName)
    {
        if (string.IsNullOrWhiteSpace(value))
            throw new ArgumentException("Значение обязательно", fieldName);
    }
}
```

Что здесь меняется по сравнению с основным вариантом:

	Основной вариант: поля + методы	**Альтернатива: автосвойства**
Чтение данных	`student.GetAge()`	`student.Age`

	Основной вариант: поля + методы	**Альтернатива: автосвойства**
Изменение данных	`student.SetAge(20)`	`student.SetAge(20)` (сеттер закрыт)
Где хранится значение	явное поле `private int age;`	неявное поле создаёт компилятор
Что защищает данные	`private` у поля	`private set` у аксессора

Оба варианта проверяются одинаково: `student.age = -3;` и `student.Age = -3;` не должны компилироваться.

Задание 2

Индивидуально. Ответьте **одной строкой кода** на каждый вопрос и напишите, что произойдёт:

1. Как объявить поле, доступное только внутри класса?
2. Как объявить тип, доступный во всей сборке, но не за её пределами?
3. Как объявить вспомогательный тип, доступный только в одном файле?
4. Как объявить метод, доступный наследникам класса?
5. Что случится при попытке унаследоваться от типа, помеченного `file`? (ответьте текстом в 1 предложение)
6. Какой принцип SOLID нарушен, если `Catalog` внутри себя создаёт `new FileProductRepository("products.csv")`?
7. Какой принцип SOLID нарушен, если `Product` умеет и хранить данные, и сохранять их в файл, и печатать отчёт?

Полную таблицу пяти принципов см. в `P3/PRACTICE 3.md`, раздел «Все принципы SOLID».

Ответы к заданию 2 (для самопроверки)

1. `private int age;` — члены класса без модификатора и так `private`, но лучше писать явно.
2. `internal class Product { }` — тип верхнего уровня по умолчанию тоже `internal`.
3. `file static class PriceFormatter { }` — модификатор `file` применим только к типам.
4. `protected void SetPrice(int price) { }` — доступно в самом классе и в наследниках.

5. Из другого файла тип с модификатором `file` не виден, поэтому наследник не скомпилируется: компилятор сообщит, что такой тип не найден.
6. **DIP**: зависимость идёт от конкретного класса, а не от интерфейса `IProductRepository`.
7. **SRP**: у класса несколько причин для изменения — данные, хранение и вывод.

Задание 3 (групповое)

Проект

Мы **продолжаем проект маркетплейса** с занятия 2. В нём уже есть `Product`, `CartItem`, `Cart`, `PriceCalculation` и `OrderService` (или аналог). На этом занятии добавляем **каталог товаров**, **инкапсуляцию** и **слой доступа к данным**.

Т.е. делаем **хранилище товаров** и инструменты **просмотра** и **изменения состава товаров в** этом **хранилище**.

Правила проекта:

1. Пространства имён фиксированы: `Marketplace.Domain` (сущности), `Marketplace.DataAccess` (хранение), `Marketplace.Presentation` (вывод).
2. Один тип — один файл.
3. Снаружи типов — только то, что действительно является контрактом; всё остальное — `private`, `internal` или `file`.

Как распределены типы проекта:

Тип	Пространство имён	Модификатор	Почему
`Product`	`Marketplace.Domain`	`public`	сущность нужн и приложению, слою хранения
`Catalog`	`Marketplace.Domain`	`public`	публичный API каталога, но коллекция внутри закрыта
`IProductRepository`	`Marketplace.DataAccess`	`public`	абстракция, от которой завися остальные (DIP
`FileProductRepository`	`Marketplace.DataAccess`	`internal`	деталь реализации, снаружи сборки не нужна

Тип	Пространство имён	Модификатор	Почему
`CatalogPrinter`	`Marketplace.Presentation`	`public`	точка вывода для приложени
`PriceFormatter`	`Marketplace.Presentation`	`file`	вспомогательн тип одного фай

Распределение задач по участникам

Участник 1 — сущность `Product` (`Product.cs`)

Требуется: товар из занятия 2 приводим к «взрослому» виду — поля закрываем, проверки собираем в одном месте.

1. Все поля — **закрытые**: идентификатор, название, цена за единицу.
2. Конструктор проверяет данные и бросает исключение при недопустимых значениях:
   - идентификатор больше нуля;
   - название не пустое;
   - цена не отрицательная.
3. Открытые методы: `GetId()`, `GetTitle()`, `GetPrice()`, `SetPrice(int)`, `GetInfo()`.
4. **1-2 приватных** метода проверки, например `EnsureTitle(string title)` и `EnsurePrice(int price)`.

```
namespace Marketplace.Domain;

public class Product
{
    //TODO: ⚠️ Закрытые поля: id, title, price ⚠️

    public Product(int id, string title, int price)
    {
        //TODO: ⚠️ Проверки и присвоение полей ⚠️
    }

    //TODO: ⚠️ public int GetId() ⚠️
    //TODO: ⚠️ public string GetTitle() ⚠️
    //TODO: ⚠️ public int GetPrice() ⚠️
    //TODO: ⚠️ public void SetPrice(int newPrice) с проверкой ⚠️
    //TODO: ⚠️ public string GetInfo() — одна строка о товаре ⚠️

    //TODO: ⚠️ private static void EnsureTitle(string value) ⚠️
    //TODO: ⚠️ private static void EnsurePrice(int value) ⚠️
}
```

Что показать в отчёте: `product.price = -100;` не компилируется, а `product.SetPrice(-100)` выбрасывает исключение.

Участник 2 — агрегирующий класс `Catalog` (`Catalog.cs`)

Требуется:

1. Закрытая коллекция товаров: `private readonly List<Product> products = new();`
2. Открытые методы:
   - `Add(Product product)` — не принимать `null` и не дублировать товары (сравнение по названию без учёта регистра и пробелов по краям);
   - `Remove(string title)` — возвращать `bool`;
   - `Find(string title)` — возвращать `Product?`;
   - `GetProducts()` — возвращать **копию** коллекции как `IReadOnlyList<Product>`;
   - `GetCount()` — количество товаров.
3. **1-2 приватных** метода: `Normalize(string title)`, `SameTitle(string left, string right)`.

```
namespace Marketplace.Domain;

public class Catalog
{
    private readonly List<Product> products = new();

    public void Add(Product product)
    {
        //TODO: ⚠️ Проверить product на null и на дубликат по названию ⚠️
    }

    //TODO: ⚠️ public bool Remove(string title) ⚠️
    //TODO: ⚠️ public Product? Find(string title) ⚠️

    // Возвращаем копию: снаружи нельзя изменить внутренний список
    public IReadOnlyList<Product> GetProducts() => products.ToList();

    //TODO: ⚠️ public int GetCount() ⚠️

    //TODO: ⚠️ private static string Normalize(string title) ⚠️
    //TODO: ⚠️ private static bool SameTitle(string left, string right) ⚠️
}
```

Что показать в отчёте: `catalog.GetProducts().Clear();` не компилируется — метода `Clear` у `IReadOnlyList<T>` нет.

Участник 3 — доступ к данным: пространство имён `Marketplace.DataAccess` (`DataAccess/`)

Требуется:

1. Объявить **открытый интерфейс** (DIP):

```
using Marketplace.Domain;

namespace Marketplace.DataAccess;

public interface IProductRepository
{
    void Save(IEnumerable<Product> products);
    List<Product> Load();
}
```

2. Реализовать его в **внутреннем** классе — деталь реализации не обязана быть видна снаружи сборки (**пример реализации репозитория на базе файла** - см. в конце этих материалов):

```
using Marketplace.Domain;

namespace Marketplace.DataAccess;

internal class FileProductRepository : IProductRepository
{
    private readonly string filePath;

    public FileProductRepository(string filePath) => this.filePath =
filePath;

    public void Save(IEnumerable<Product> products)
    {
        //TODO: ⚠️ Сохранить каждый товар строкой в файл ⚠️
        //TODO: ⚠️ Использовать ПРИВАТНЫЙ метод Serialize ⚠️
    }

    public List<Product> Load()
    {
        //TODO: ⚠️ Прочитать строки файла и собрать List<Product> ⚠️
        //TODO: ⚠️ При отсутствии файла вернуть пустой список ⚠️
        //TODO: ⚠️ Использовать ПРИВАТНЫЙ метод Deserialize ⚠️
        return new List<Product>();
```

```
    }

    //TODO: ⚠️ private static string Serialize(Product product) ⚠️
    //TODO: ⚠️ private static Product Deserialize(string line) ⚠️
}
```

3. Формат хранения выберите сами (например, `1;Кружка;450` — по строке на товар) и **опишите его** в комментарии к классу.
4. Не забудьте `using Marketplace.Domain;` — типы слоя хранения берутся из домена.

Участник 4 — вывод и отчёты (SRP)

Требуется:

1. Класс `CatalogPrinter` (`CatalogPrinter.cs`) — **только** печать: `PrintAll(IEnumerable<Product> products)`, `PrintTop(IEnumerable<Product> products, int count)`; внутри 1-2 приватных метода подготовки строки.
2. В том же файле — `file static class PriceFormatter` с методом `Format(int price)` (демонстрация файла как области видимости).
3. Отдельный класс `CatalogReport` (`CatalogReport.cs`) — **только** расчёты: `GetAveragePrice(IEnumerable<Product> products)` и `GetSummary(IEnumerable<Product> products)`; ни печати, ни работы с файлами.
4. Убедиться, что ни `Product`, ни `Catalog` не занимаются печатью — это и есть SRP.

```
using Marketplace.Domain;

namespace Marketplace.Presentation;

public class CatalogPrinter
{
    public void PrintAll(IEnumerable<Product> products)
    {
        //TODO: ⚠️ Напечатать все товары через PrepareLine ⚠️
    }

    public void PrintTop(IEnumerable<Product> products, int count)
    {
        //TODO: ⚠️ Напечатать count самых дорогих товаров ⚠️
    }

    //TODO: ⚠️ private string PrepareLine(Product product) ⚠️
}

// Виден только в этом файле
```

```
file static class PriceFormatter
{
    public static string Format(int price) => $"{price} y.e.";
}
```

Вопрос для отчёта: почему `PriceFormatter` не виден в файле `CatalogReport.cs` и что пришлось бы сделать, если бы он понадобился в двух файлах?

> **Ответ:** модификатор `file` ограничивает тип одним файлом; для двух файлов нужен `internal` (внутри сборки) или `public` (для всех).

Компоновка (задача Лидера)

Лидер рабочей группы:

1. собирает проект: по одному типу на файл, пространства имён `Marketplace.Domain`, `Marketplace.DataAccess`, `Marketplace.Presentation`;
2. пишет точку входа, где:
   - создаёт 3-4 товара, в том числе с недопустимыми данными (в `try/catch`) — чтобы показать защиту целостности;
   - добавляет их в `Catalog`, ищет, удаляет, выводит;
   - сохраняет и загружает товары через `IProductRepository` **через интерфейс** (DIP);
   - печатает отчёт через `CatalogPrinter` и `CatalogReport`;
3. проверяет, что изменить коллекцию каталога снаружи нельзя;
4. оформляет отчёт.

Пример точки входа для компоновки:

```
using Marketplace.DataAccess;
using Marketplace.Domain;
using Marketplace.Presentation;

var catalog = new Catalog();
catalog.Add(new Product(1, "Кружка", 450));
catalog.Add(new Product(2, "Термос", 1200));
catalog.Add(new Product(3, "Набор стикеров", 90));

// Целостность данных: недопустимые значения не попадают в объект
try
{
    catalog.Add(new Product(4, "   ", -5));
}
catch (ArgumentException ex)
```

```
{
    Console.WriteLine($"Данные отклонены: {ex.Message}");
}

// DIP: работаем через интерфейс, а не через конкретный класс
IProductRepository repository = new FileProductRepository("products.csv");
repository.Save(catalog.GetProducts());

var loaded = repository.Load();
Console.WriteLine($"Загружено товаров: {loaded.Count}");

// SRP: печать и расчёт — в разных типах
var printer = new CatalogPrinter();
printer.PrintAll(loaded);
printer.PrintTop(loaded, 2);

Console.WriteLine(new CatalogReport().GetSummary(loaded));

// Коллекцию каталога снаружи не изменить:
// catalog.GetProducts().Clear();   // ошибка: у IReadOnlyList<T> нет Clear
```

Рекомендации по выполнению

1. **Не меняйте общий контракт в одиночку.** Названия пространств имён, интерфейсов и методов фиксируются в начале работы всей группой.
2. **Проверяйте доступ на практике:** попробуйте обратиться к закрытому полю — и покажите в отчёте текст ошибки компилятора. Это лучшее доказательство, что инкапсуляция работает.
3. **Проверяйте данные в одном месте** (в конструкторе или в приватном `Ensure...`), а не в каждом методе по-своему.
4. **DIP проверяется просто:** в точке входа переменная объявлена как `IProductRepository repo = new FileProductRepository("products.csv");` — если заменить файловый репозиторий на «в памяти», остальной код не меняется.
5. **Каждый участник** пишет либо небольшой класс, либо как минимум один серьёзный публичный метод сервиса с 1-2 приватными методами внутри.

Требования к отчёту

Отчёт оформляет **Лидер группы**, файл — в формате Markdown.

1. **Титульная часть:** номер занятия, тема, состав рабочей группы (3-4 человека, лидер отмечен).
2. **Текст исходного задания** — полностью, как он приведён в этом файле.
3. **Для каждого задания:** краткое описание выполненного решения (2-5 предложений: что сделано, какие модификаторы доступа использованы и почему).
4. **Под каждым описанием — листинг решения** (код с комментариями).
5. **Таблица модификаторов:** для каждого типа проекта укажите выбранный модификатор и обоснование (**публичный API** или **деталь реализации**).
6. **Доказательство работы DIP:** фрагмент кода точки входа с переменной типа интерфейса.
7. **Доказательство работы инкапсуляции:** текст ошибки компилятора при обращении к закрытому полю и при попытке изменить коллекцию каталога.
8. **Список вопросов** от каждого участника группы.
9. **Выводы:** где пригодились `internal` и `file`, что изменилось бы без инкапсуляции.

Критерии оценки группы:

- проект собирается и запускается без правок;
- данные нельзя испортить снаружи;
- сохранение и загрузка работают;
- зависимости идут через интерфейс, печать вынесена из сущностей;
- каждый участник написал свою часть;
- оформление отчёта соответствует требованиям выше.

Срок сдачи — до вечера пятницы.

Материалы

План занятия

Виды модификаторов доступа

Модификатор доступа определяет, **кто** может использовать тип или его член.

Зачем это нужно:

- **защита данных** — снаружи нельзя испортить состояние объекта;
- **свобода внутри** — реализацию можно менять, не ломая тех, кто пользуется классом;

- **явный контракт** — по описанию класса видно, что для внешнего мира важно, а что — деталь.

Полная таблица

Модификатор	Где доступен	Где обычно применяется
`public`	везде, без ограничений	публичный API типа
`private`	только внутри своего типа	поля, вспомогательные методы
`protected`	внутри своего типа и в наследниках	члены для расширения через наследование
`internal`	внутри той же **сборки**	«внутренний» код проекта
`protected internal`	наследники **ИЛИ** любая точка сборки	реже, «для своих и для наследников»
`private protected`	наследники **И** только внутри сборки	«для наследников, но не из внешних сборок»
`file` (C# 11)	только в пределах **своего файла**	локальные вспомогательные типы

Значения по умолчанию:

- тип верхнего уровня (`class` , `struct` , `interface` , ...) без модификатора — **`internal`** ;
- член класса без модификатора — **`private`** ;
- член интерфейса без модификатора — **`public`** .

Вопрос для закрепления: какой модификатор доступа получит поле `int count;` , если явно его не указать?

> **Ответ:** `private` — для членов класса это значение по умолчанию.

Пример: инкапсуляция в классе студента

```
public class Student
{
    // Закрытые поля: снаружи их нельзя испортить напрямую
    private string name;
    private int age;

    public Student(string name, int age)
    {
        this.name = name;
        this.age = age;
```

```
    }

    // Открытые методы — единственный способ взаимодействия
    public string GetName() => name;

    public int GetAge() => age;

    public void SetAge(int newAge)
    {
        if (newAge < 14 || newAge > 100)
            throw new ArgumentOutOfRangeException(nameof(newAge),
"Недопустимый возраст");

        age = newAge;   // проверенное значение
    }
}
```

Если сделать `age` открытым, любой код сможет записать `student.age = -5`, и объект станет «некорректным».

Вопрос для закрепления: что произойдёт с клиентским кодом, если мы заменим `SetAge` на `SetBirthDate`?

> **Ответ:** он перестанет компилироваться: все вызовы `SetAge(...)` станут ошибкой. Правильнее оставить `SetAge` и внутри пересчитывать дату рождения либо заранее договориться о новом контракте.

`protected` — доступ для наследников

```
public class Student
{
    protected int course;   // доступно в наследниках

    public int GetCourse() => course;
}

public class GraduateStudent : Student
{
    public void MoveToCourse(int newCourse) => course = newCourse;   //
можно
}
```

Из объекта `Student` снаружи обратиться к `course` нельзя: `student.course` не скомпилируется.

Вопрос для закрепления: можно ли объявить `protected` поле в структуре?

> **Ответ:** нет. Структура не поддерживает наследование, поэтому `protected`, `protected internal` и `private protected` для её членов запрещены — компилятор выдаст ошибку.

Модификаторы доступа в рамках проекта

Проект (`.csproj`) обычно собирается в **одну сборку**. Пока у нас небольшая программа, «проект» и есть «сборка».

Модификатор **`internal`** открывает доступ **всем типам проекта**, но закрывает его для программ, которые будут подключать нашу сборку.

```
// Файл JournalService.cs
internal class JournalService
{
    public void AddMark(string subject, int mark) { /* ... */ }
}
```

Внутри этого же проекта:

```
internal class Program
{
    private static void Main()
    {
        var service = new JournalService();   // доступно: та же сборка
        service.AddMark("ООП", 5);
    }
}
```

В другом проекте, где подключена наша сборка, тип `JournalService` **не виден**.

Вопрос для закрепления: `internal` строже или мягче, чем `public`?

> **Ответ:** строже: `public` виден всем сборкам, а `internal` — только своей.

Пограничный случай — **несогласованная доступность**: открытый метод не может возвращать закрытый тип.

```
internal class Journal
{
```

```
    // Ошибка компиляции: метод public, а тип Journal — internal
    // public Journal Create() => new Journal();
}
```

Вопрос для закрепления: почему компилятор запрещает открытому методу возвращать внутренний тип?

> **Ответ:** это несогласованная доступность: вызывающий код из другой сборки получил бы значение типа, о котором ничего не знает, и не смог бы им воспользоваться.

Модификаторы доступа в рамках сборки

Сборка — это собранный файл `.dll` или `.exe`, который подключают другие программы.

Правила простыми словами:

- за пределы сборки «видны» только типы и члены с **`public`**;
- **`internal`** (и по умолчанию `protected internal` / `private protected`) — только внутри;
- **`private`**, **`protected`** — всегда остаются внутри типа/иерархии.

```
// Проект Campus.Core (сборка Campus.Core.dll)
public class University
{
    private readonly List<Student> students = new();

    public void Enroll(Student student) => students.Add(student);   // виден
снаружи
    internal void Archive() { students.Clear(); }                   //
только внутри сборки
}
```

```
// Проект Campus.App (другая сборка, ссылается на Campus.Core)
var university = new University();
university.Enroll(new Student("Анна", 20));   // OK
// university.Archive();                      // ошибка: internal не виден
```

Дружественные сборки. Иногда тестам или внутренним приложениям нужно видеть `internal`:

```
<!-- Campus.Core.csproj -->
<ItemGroup>
  <InternalsVisibleTo Include="Campus.Tests" />
</ItemGroup>
```

Такой элемент проекта добавляет в сборку атрибут `InternalsVisibleTo`: тип остаётся `internal`, но указанная сборка (`Campus.Tests`) видит его так же, как своя.

Вопрос для закрепления: чем `protected internal` отличается от `private protected`?

> **Ответ:** `protected internal` — это «ИЛИ»: доступ есть у наследников **вне** сборки и у любого кода **внутри** сборки. `private protected` — это «И»: доступ только у наследников, и только внутри своей сборки.

Модификатор	Наследник вне сборки	Код внутри сборки, но не наследник
`protected internal`	видит	видит
`private protected`	не видит	не видит

Файл как область видимости

С C# 11 появился модификатор `file`: тип виден **только в том файле**, где объявлен.

```
// JournalFormatter.cs
file static class JournalFormatter
{
    public static string Format(int mark) => mark == 0 ? "не сдано" : $"
{mark} балл(ов)";
}

public class Journal
{
    public string Print(int mark) => JournalFormatter.Format(mark);   // OK:
тот же файл
}
```

В другом файле:

```
// Program.cs
// JournalFormatter.Format(5);   // ошибка: тип виден только в своём файле
```

Когда это удобно:

- вспомогательный тип нужен **одному** файлу;
- нет смысла засорять пространство имён проекта;
- нужно гарантированно **не дать** использовать деталь реализации снаружи.

Ограничения: `file` объявляется **только для типов** (не для членов), тип не может быть `public`, и использовать его из другого файла невозможно даже через `InternalsVisibleTo`.

Вопрос для закрепления: что даёт модификатор `file`, если в проекте всего один файл?

> **Ответ:** почти ничего: тип и так не выйдет за пределы файла. Польза появляется, когда файлов много — не нужно придумывать уникальные имена вспомогательных типов.

Пространства имён в проекте

Пространства имён мы разбирали на занятии 2. На занятии 3 они важны по двум причинам:

1. **Порядок в проекте.** Типы раскладываются по «папкам», и по имени пространства имён видно назначение типа.
2. **Связь с доступом.** `internal` и `file` ограничивают видимость внутри сборки или файла, а пространство имён только **группирует** типы и **не** ограничивает доступ.

```
// Campus.Domain/Student.cs
namespace Campus.Domain;

public class Student { /* ... */ }
```

```
// Campus.DataAccess/FileStudentRepository.cs
namespace Campus.DataAccess;

internal class FileStudentRepository : IStudentRepository { /* ... */ }
```

```
// Campus.Presentation/StudentPrinter.cs
using Campus.Domain;          // подключаем типы другой «папки»

namespace Campus.Presentation;

public class StudentPrinter
{
    public void Print(Student student) =>
Console.WriteLine(student.GetFullName());
}
```

Вопрос для закрепления: защищает ли пространство имён `Campus.Domain` тип `Student` от использования в другом проекте?

> **Ответ:** нет. Доступ ограничивают модификаторы (`internal`, `private`), а пространство имён только группирует типы и убирает конфликты имён.

Проект занятия: маркетплейс

С занятия 2 мы ведём один сквозной проект — **интернет-маркетплейс**. Модификаторы доступа в нём распределяются так:

Тип	Пространство имён	Модификатор	Роль
`Product`	`Marketplace.Domain`	`public`	сущность, которой пользуются и приложение, и хранение
`Catalog`	`Marketplace.Domain`	`public`	публичный API каталога, коллекция товаров внутри закрыта
`IProductRepository`	`Marketplace.DataAccess`	`public`	абстракция для DIP (см. ниже)
`FileProductRepository`	`Marketplace.DataAccess`	`internal`	деталь реализации, снаружи сборки не нужна

Тип	Пространство имён	Модификатор	Роль
`CatalogPrinter`	`Marketplace.Presentation`	`public`	вывод для приложения
`PriceFormatter`	`Marketplace.Presentation`	`file`	вспомогательн тип одного фай

```
// Marketplace.Domain/Product.cs
namespace Marketplace.Domain;

public class Product
{
    private int price;                // деталь состояния — снаружи не
видна

    public int GetPrice() => price;    // контракт

    public void SetPrice(int value)
    {
        if (value < 0)
            throw new ArgumentOutOfRangeException(nameof(value), "Цена не
может быть отрицательной");

        price = value;
    }
}
```

```
// Marketplace.DataAccess/FileProductRepository.cs
namespace Marketplace.DataAccess;

internal class FileProductRepository : IProductRepository   // деталь
реализации
{
    private readonly string filePath;

    public FileProductRepository(string filePath) => this.filePath =
filePath;

    // ... Save и Load
}
```

Вопрос для закрепления: почему `Product` и `Catalog` открытые, а `FileProductRepository` — внутренний?

> **Ответ:** `Product` и `Catalog` — часть контракта приложения, ими пользуются разные слои; `FileProductRepository` — конкретная деталь хранения, и снаружи сборки её знать не должны.

Как это связано с SOLID

Модификаторы доступа — это **инструмент**, а не цель. В лабораторной работе нас особенно интересуют два принципа, но знать нужно все пять.

Все принципы SOLID

Принцип	Расшифровка	Смысл в одном предложении	Признак нарушения
S — SRP	Single Responsibility Principle, единственная ответственность	У типа одна причина для изменения, одна задача.	Класс `Student` умеет и хранить данные, и печатать отчёт, и сохранять файл.
O — OCP	Open/Closed Principle, открытость/закрытость	Тип открыт для расширения, но закрыт для изменения.	Чтобы добавить новый вид отчёта, приходится править `switch` внутри сервиса.
L — LSP	Liskov Substitution Principle, подстановка Барбары Лисков	Наследник заменяет базовый тип, не ломая поведение.	`GraduateStudent` бросает исключение там, где `Student` работает нормально.
I — ISP	Interface Segregation Principle, разделение интерфейсов	Много маленьких интерфейсов лучше одного большого.	Интерфейс `IStudentRepository` требует `Save`, `Load`, `Export`, `SendEmail`, хотя реализации нужен только `Load`.
D — DIP	Dependency Inversion Principle, инверсия зависимостей	Зависим от абстракции, а не от конкретной реализации.	`University` создаёт внутри себя `new FileStudentRepository("students.csv")`

Для занятия 3 главные — **SRP** и **DIP**: они напрямую проверяются в домашнем задании.

- **SRP** (единственная ответственность). Каждый тип отвечает за одну задачу: `Student` хранит и проверяет свои данные, `StudentsRepository` сохраняет их, `StudentPrinter` печатает. Если класс делает всё сразу, правильные модификаторы доступа уже не помогут.
- **DIP** (инверсия зависимостей). Код должен зависеть от **абстракции**, а не от конкретной реализации.

```
public interface IStudentsRepository
{
    void Save(IEnumerable<Student> students);
    IReadOnlyList<Student> Load();
}

internal class FileStudentsRepository : IStudentsRepository
{
    private readonly string path;

    public FileStudentsRepository(string path) => this.path = path;

    public void Save(IEnumerable<Student> students) { /* запись в файл */ }

    public IReadOnlyList<Student> Load() { /* чтение из файла */ }
}
```

`University` теперь зависит от `IStudentsRepository` и **не знает**, файл это, база данных или память.

Вопрос для закрепления: зависимость от интерфейса или от класса лучше и почему?

> **Ответ:** от интерфейса: реализацию (файл, база данных, память, заглушка для теста) можно заменить, не меняя код, который её использует.

Мини-упражнение на закрепление

1. Создайте в одном файле `Journal.cs` класс `Journal` с закрытым полем `marks` (список оценок) и открытыми методами `Add` и `GetAverage`.
2. В том же файле объявите `file static class MarkHelper` с методом форматирования оценки.

3. В `Program.cs` попробуйте обратиться к `MarkHelper` и объясните ошибку компилятора.
4. Замените у `Journal` модификатор на `internal`, а затем на `public` и обсудите, кто теперь может им пользоваться.

Вопросы для самопроверки (ответ — одна строка кода)

1. Как сделать поле доступным только внутри своего класса?

   > `private int count;`

2. Как объявить класс, видимый во всём проекте, но не за его пределами?

   > `internal class Journal { }`

3. Как ограничить вспомогательный класс одним файлом?

   > `file static class MarkHelper { }`

4. Как объявить интерфейс, которым смогут пользоваться все сборки решения?

   > `public interface IStudentRepository { }`

Что запомнить:

- `public` — контракт для всех, `internal` — «для своей сборки», `file` — «для своего файла»;
- `private` защищает состояние, `protected` отдаёт его наследникам;
- открытый метод не может отдавать наружу закрытый тип;
- пространство имён группирует типы, но **не** ограничивает доступ;
- в проекте занятия: `Product` и `Catalog` — `public`, `FileProductRepository` — `internal`, `PriceFormatter` — `file`;
- SRP — одна причина изменения у типа, DIP — зависимость от интерфейса, а не от класса.

Иллюстрация DIP на примере репозитория Студентов

Сделаем свою "базу данных" из одной "таблицы" - репозиторий записей о Студентах, который в частном случае будет хранить записи о студентах в CSV файле на диске.

1. Модель `Student`

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public int Course { get; set; }
    public DateTime BirthDate { get; set; }
    public int Age { get; set; }
}
```

> Обратите внимание: я назвал поле `BirthDate` (исправил опечатку `bitrthDate`), но суть та же.

2. Абстракция для DIP — интерфейс `IStudentRepository`

Это и есть та самая «абстракция». Высокоуровневый код (например, консольное меню или сервис) будет зависеть только от неё, а не от файловой реализации.

```
public interface IStudentRepository
{
    IEnumerable<Student> GetAll();
    Student? GetById(int id);
    void Add(Student student);
    void Update(Student student);
    void Delete(int id);
}
```

3. Реализация — `CsvStudentRepository`

Файл `student.csv` лежит рядом с приложением. Формат строки: `Id,Name,Course,BirthDate,Age`.

```
using System.Globalization;

public class CsvStudentRepository : IStudentRepository
{
    private readonly string _filePath;

    public CsvStudentRepository(string filePath = "student.csv")
    {
        _filePath = filePath;
        if (!File.Exists(_filePath))
            File.WriteAllText(_filePath, string.Empty); // создаём пустой
```

файл

```
    }

    public IEnumerable<Student> GetAll()
    {
        if (!File.Exists(_filePath))
            return Enumerable.Empty<Student>();

        var lines = File.ReadAllLines(_filePath)
                        .Where(l => !string.IsNullOrWhiteSpace(l));

        var students = new List<Student>();
        foreach (var line in lines)
        {
            var parts = line.Split(',');
            if (parts.Length != 5) continue; // пропускаем битые строки

            students.Add(new Student
            {
                Id = int.Parse(parts[0]),
                Name = parts[1],
                Course = int.Parse(parts[2]),
                BirthDate = DateTime.ParseExact(parts[3], "yyyy-MM-dd",
CultureInfo.InvariantCulture),
                Age = int.Parse(parts[4])
            });
        }
        return students;
    }

    public Student? GetById(int id) => GetAll().FirstOrDefault(s => s.Id ==
id);

    public void Add(Student student)
    {
        var students = GetAll().ToList();
        student.Id = students.Any() ? students.Max(s => s.Id) + 1 : 1;
        students.Add(student);
        SaveAll(students);
    }

    public void Update(Student student)
    {
        var students = GetAll().ToList();
        var index = students.FindIndex(s => s.Id == student.Id);
        if (index == -1) return; // или бросить исключение
```

```
        students[index] = student;
        SaveAll(students);
    }

    public void Delete(int id)
    {
        var students = GetAll().Where(s => s.Id != id).ToList();
        SaveAll(students);
    }

    private void SaveAll(IEnumerable<Student> students)
    {
        var lines = students.Select(s =>
            $"{s.Id},{s.Name},{s.Course},{s.BirthDate:yyyy-MM-dd},{s.Age}");
        File.WriteAllLines(_filePath, lines);
    }
}
```

4. Пример использования (клиент зависит от интерфейса)

Здесь видно главное: `Main` работает с `IStudentRepository`, а не с `CsvStudentRepository`. Это и есть инверсия зависимостей.

```
class Program
{
    static void Main()
    {
        // Зависимость "инвертирована": клиент знает только интерфейс.
        IStudentRepository repo = new CsvStudentRepository("student.csv");

        repo.Add(new Student
        {
            Name = "Иван",
            Course = 2,
            BirthDate = new DateTime(2004, 5, 12),
            Age = 21
        });

        repo.Add(new Student
        {
            Name = "Мария",
            Course = 1,
            BirthDate = new DateTime(2005, 9, 3),
            Age = 20
        });
```

```
        Console.WriteLine("Все студенты:");
        foreach (var s in repo.GetAll())
            Console.WriteLine($"{s.Id}. {s.Name}, курс {s.Course}, возраст
{s.Age}");

        // Обновление
        var student = repo.GetById(1);
        if (student != null)
        {
            student.Course = 3;
            student.Age = 22;
            repo.Update(student);
        }

        // Удаление
        repo.Delete(2);

        Console.WriteLine("\nПосле изменений:");
        foreach (var s in repo.GetAll())
            Console.WriteLine($"{s.Id}. {s.Name}, курс {s.Course}, возраст
{s.Age}");
    }
}
```

Что тут важно для DIP

- **`IStudentRepository`** — абстракция.
- **`CsvStudentRepository`** — деталь реализации (работа с файлом).
- **`Program`** — высокоуровневый модуль, который зависит только от `IStudentRepository`.

Если завтра вы захотите хранить студентов в SQLite или в памяти — достаточно написать `InMemoryStudentRepository : IStudentRepository` и подставить его в `Main`, **не меняя** код клиента. Это и есть смысл «абстракции для DIP».

В примере нет обработки ошибок и экранирования особых символов в строках.